–实验一 基于华为鲲鹏云服务器的 ARM 开发

一、实验目的

1. 掌握在鲲鹏云服务器上搭建 ARMv8 汇编实验环境。
2. 掌握基本的汇编程序编写；
3. 了解 ARMv8 开发编译环境的配置以及 ARM 平台；
4. 在华为鲲鹏云服务器上，编译运行 3 个不同功能的示例程序；
5. 掌握在 ARM 平台上实现 C 语言代码中内嵌汇编代码的方法。

二、实验内容

1. 实现 ARM 平台精简指令集（RISC）编写的 hello-world 程序的编译和运行。
2. 在华为 CloudIDE 开发平台上实现 hello-world 程序的编译和运行
3. 实现 ARM 平台上通过 C 语言源码来调用汇编源码中的代码。
4. 实现在 ARM 平台上通过 C 语言代码内嵌汇编代码的方式，将一个整数类型值，以字节为单位从小尾端转到大尾端或者相反的功能。

三、实验原理

1. hello-world 示例程序

在本例子中，两次使用软中断指令 svc 来进行系统调用，系统调用号通过 x8 寄存器传递。在第一次使用 svc 指令来在屏幕上打印一个字符串“Hello”：x0 寄存器用于存放标准屏幕输出 stdout 描述符 0，表明将向屏幕输出一些内容；x1 寄存器用于存放待输出的字符串的首地址 msg；x2 寄存器用于存放待输出字符串的长度 len；x8 寄存器用于存放系统功能调用号 64，即 64 号系统功能即系统写功能 sys_write()，写的目标在 x0 中定义；svc #0 表示是一个系统功能调用。

第二次使用 svc 指令来退出当前程序：x0 寄存器用于存放退出操作码 123，不同的退出操作码将对应不同的退出操作；x8 寄存器用于存放系统功能调用号 93，即 93 号系统功能即系统退出功能 sys_exit()，退出操作码在 x0 中定义；svc #0 表示是一个系统功能调用。

注意：像这种系统功能调用的方式和功能号，都是基于 Arm64 处理器体系结构以及之上所运行的 linux kernel 甚至 BIOS 来共同支持，而不仅仅是 Arm64 架构自身所能完成的。

在.data 部分，加载 msg 和 len 实际上使用的是文字池的方法，即将变量地址放在代码段中不会执行到的位置（因为第二次使用 svc 指令来退出当前程序之后，是不可能将 svc #0 指令之后的内容来当做指令加以执行的），使用时先加载变量的地址，然后通过变量的地址得到变量的值。

本代码是 Aarch64 体系结构的汇编代码，需要在 ArmV8 处理器上运行。寄存器 Xn 都是 Aarch64 体系结构中的寄存器，svc 是 Aarch64 体系结构中的指令。

2. 在华为 CloudIDE 上实现 Helloworld 示例程序

CloudIDE 是面向云原生的轻量级 WebIDE，通过浏览器访问即可实现云端开发环境获取、代码编写、编译调试、运行预览、访问代码仓库、命令行执行等能力，同时支持丰富的插件扩展。可以为开发者提供轻量极速的在线编程体验，帮助开发者快速可靠交付代码,并打通整个开发、测试和运行时。CloudIDE 是按需计费服务，根据用户使用的计算和存储资源的数量和时长按需计费，开通服务后不创建 IDE 实例不会扣费。

目前 CloudIDE 提供了两种 CPU 架构的 IDE 实例：x86 和鲲鹏（AArch64），其中鲲鹏实例资源免费，x86 实例资源按需付费。

3. 通过 C 语言源码来调用汇编源码

该汇编代码是针对Aarch64架构的。在汇编程序中，用.global定义一个全局函数strcpy1，然后该函数就可以在 C 代码中用 extern 关键字加以声明，然后直接调用。

4. C 语言代码内嵌汇编代码

通过 C 语言代码内嵌汇编代码，将一个整数类型值，以字节为单位从小尾端转到大尾端或者相反的功能。例如小尾端时 32bit 整数值用 16 进制表示为 0x12345678，将其以字节为单位转换为大尾端存储后，该值为 0x78563412。

四、实验步骤

1. hello-world 示例程序

以下步骤以在华为鲲鹏云服务器上执行为例。

- 创建 hello 目录

执行以下命令，创建 hello 目录，存放该程序的所有文件，并进入 hello 目录。

```
mkdir hello
cd hello
```

- 创建示例程序源码 hello.s
- 执行以下命令，创建示例程序源码 hello.s。

```
vim hello.s
```

- 代码内容如下：

```
.text
.global tart1
tart1:
        mov x0,#0
        ldr x1,=msg
        mov x2,len
        mov x8,64
        svc #0

        mov x0,123
        mov x8,93
        svc #0

.data
msg:
        .ascii "Hello World!\n"
len=.-msg
```

- 进行编译运行，保存示例源码文件，然后退出 vim 编辑器。在当前目录中依次执以下命令，进行代码编译运行。

```
as hello.s –o hello.o
ld hello.o –o hello
./hello
```

- 通过上述代码运行，可以看到，编写的 hello-wolrd 示例程序已经在华为鲲鹏云服务器上通过编译和运行，并成功输出结果。

2. 在华为 CloudIDE 上实现 Helloworld 示例程序

按照教程进入华为云官网的 CloudIDE 开发平台，按照 1 的流程进行代码边写即可

3. 使用 C 语言源码来调用汇编源码

- 执行以下命令，创建 called 目录存放该程序的所有文件，并进入 called 目录。

```
mkdir called
cd called
```

- 执行以下命令，创建示例调用 C 语言程序源码 globalCalling.c。

```
vim globalCalling.c
```

- 代码内容如下：

```
/* globalCalling.c*/
#include <stdio.h>
extern void strcpy1(char *d, const char *s);
int main()
{
    const char *srcstring="Source string";
    char dststring[]="Destination string";

    printf("Original Status: %s    %s\n",srcstring,dststring);
  strcpy1(dststring,srcstring);
  printf("Modified Status: %s    %s\n",srcstring,dststring);
    return 0;
}
```

- 执行以下代码命令，创建被调用的汇编语言程序源码 globalCalled.S。

```
vim globalCalled.S
```

- 代码内容如下：

```
/* globalCalled.S */
.global strcpy1

# Start the function: strcpy1
strcpy1:
LDRB w2,[X1],#1
STR w2,[X0],#1
CMP w2,#0    //ascii code "NUL" is the last character of a string;
BNE strcpy1
RET
```

- 保存示例源码文件，然后退出 vim 编辑器。在当前目录中依次执行以下命令，进行代码编译运行。

```
gcc globalCalling.c globalCalled.S -o called
./called
```

- 通过上述代码运行，可以看出，编写的使用 C 语言代码调用汇编程序已经在华为鲲鹏云服务器上通过编译和运行，并成功输出结果：

  Original Status: Source string   Destination string

  Modified Status: Source string   Source string

4. C 语言代码内嵌汇编代码

以下步骤以在华为鲲鹏云服务器上执行为例。

- 执行以下命令，创建 builtin 目录存放该程序的所有文件，并进入 bulitin 目录。

```
mkdir builtin
cd builtin
```

- 执行以下命令，创建 C 语言内嵌汇编程序源码 globalBuiltin.c。

```
vim globalBuiltin.c
```

- 代码内容如下：

```
/* globalBuiltin.c*/
#include <stdio.h>
int main()
{
int val=0x12345678;

  __asm__ __volatile__(
        "mov x3,%1\n"
         "mov w3,w3, ror #8\n"
         "bic w3,w3, #0x00ff00ff\n"

         "mov x4,%1\n"
         "mov w4,w4, ror #24\n"
         "bic w4,w4, #0xff00ff00\n"

         "add w3,w4,w3\n"
         "mov %0,x3\n"

         :"=r"(val)
         :"0"(val)
         :"w3","w4","cc"
         );

printf("out is %x \n",val);
return 0;
}
```

- 保存示例源码文件，然后退出 vim 编辑器。在当前目录中依次执行以下命令，进行代码编译。

```
gcc -E globalBuiltin.c -o globalBuiltin.i
gcc -S globalBuiltin.i -o globalBuiltin.s
gcc -c globalBuiltin.s -o globalBuiltin.o
gcc globalBuiltin.o -o globalBuiltin
```

- 命令和生成的文件如下：

```
[root@ecs-hw ~]# mkdir builtin
[root@ecs-hw ~]# cd builtin
[root@ecs-hw builtin]# vim globalBuiltin.c
[root@ecs-hw builtin]# [root@ecs-hw builtin]# gcc -E globalBuiltin.c -o globalBuiltin.i
[root@ecs-hw builtin]# gcc -S globalBuiltin.i -o globalBuiltin.s
[root@ecs-hw builtin]# gcc -c globalBuiltin.s -o globalBuiltin.o
[root@ecs-hw builtin]# gcc globalBuiltin.o -o globalBuiltin
[root@ecs-hw builtin]# ls
globalBuiltin  globalBuiltin.c  globalBuiltin.i  globalBuiltin.o  globalBuiltin.s
```

- 运行生成的 globalBuiltin 文件，查看输出结果。

  命令如下：

```
./globalBuiltin
```

- 结果如下：

```
[root@ecs-hw builtin]# ./globalBuiltin
out is 78563412
```

- 通过上述代码运行，可以看出，编写的 C 语言代码内嵌汇编程序已经在华为鲲鹏云服务器上通过编译和运行，并成功输出结果：out is 78563412。

五、实验结果

1. hello-world 示例程序

```
[root@ecs-xjm ~]# mkdir hello
mkdir: cannot create directory ‘hello’: File exists
[root@ecs-xjm ~]# ls
hello  hello.o  hello.s
[root@ecs-xjm ~]# cd hello
-bash: cd: hello: Not a directory
[root@ecs-xjm ~]# vim hello.s
[root@ecs-xjm ~]# as hello.s -o hello.o
[root@ecs-xjm ~]# ld hello.o -o hello
ld: warning: cannot find entry symbol _start; defaulting to 00000000004000b0
[root@ecs-xjm ~]# ./hello
Hello World!
```

2. 在华为 CloudIDE 上实现 Helloworld 示例程序

```
user@e6iyyqnjkp5moon-machine:~/arm64_demo$ as -o hello.o hello.s
user@e6iyyqnjkp5moon-machine:~/arm64_demo$ ./hello.o
bash: ./hello.o: Permission denied
user@e6iyyqnjkp5moon-machine:~/arm64_demo$ ld -o hello hello.o
ld: warning: cannot find entry symbol _start; defaulting to 00000000004000b0
user@e6iyyqnjkp5moon-machine:~/arm64_demo$ ./hello
Hello World!
```

3. 通过 C 语言源码来调用汇编源码

```
[root@ecs-xjm ~]# cd called/
[root@ecs-xjm called]# vim globalCalling.c
[root@ecs-xjm called]# vim globalCalling.S
[root@ecs-xjm called]# gcc globalCalling.c globalCalling.s -o called
gcc: error: globalCalling.s: No such file or directory
[root@ecs-xjm called]# gcc globalCalling.c globalCalling.S -o called
[root@ecs-xjm called]# ./called
Original Status: Source string   Destination string
Modified Status: Source string   Source string
```

4. C 语言代码内嵌汇编代码

```
[root@ecs-xjm builtin]# ./globalBuiltin
out is 78563412
```

六、心得体会

通过这次实验，我掌握了在云服务器上从搭建资源环境到代码输入并且运行的一系列命令流程，并且第一次接触到了云编程的使用，通过三个编程代码的编写，我掌握了通过 C 语言源码来调用汇编源码、C 语言代码内嵌汇编代码等知识，同时加深了我对汇编语言的理解。